EECS3311 Stage 1 Project Design Report

1. Project Overview

1.1 Purpose and Architecture

In game development, projects usually have a large amount of assets like 3d models, textures, materials etc,,, created with different tools, transferring these files around between platforms becomes a hassle and would be very repetitive. Especially when files are poorly organized and not named properly, fixing compatibility issues, and related models and textures may be hard to identify.

For this application, the target users would be independent game developers, student game developers and 3d artists who consistently work with assets.This AI project can help automate and assist this process, The agent can help matching which asset belongs together, rename assets, making sure the export of these assets are appropriate for the platform, migrating the asset properly and detect potential compatibility issues.

An ai agent is appropriate for this role because a lot of assets relationships and migration decisions alone cannot be determined deterministically, especially when it comes to files that are poorly named and files that may need extra conversion or adjustments before getting imported onto another platform.

This project plans to use openAI though Oauth and uses it as a migration agent to choose which tools to use within the program to ensure a seamless transfer. The actual transfer and conversion would be done with the software, the ai agent would just be the operator.

1.2 Feature Specifications

F01. Import Assets

- **Description:** Scan an asset folder.
- **GUI:** Import Project.
- **CLI:** import.
- **Input:** Folder path.
- **Output:** Inventory and warnings.
- **AI:** Deterministic.
- **Workflow:** Scan supported files, extract metadata and display the inventory.
- **Exceptions:** Reject inaccessible folders; skip and flag unsupported or corrupted files.

F02. Categorize Assets

- **Description:** Suggest categories such as characters, weapons and textures.
- **GUI:** Categorize Assets.
- **CLI:** analyze categories.
- **Input:** Names, types and metadata.
- **Output:** Reviewed categories.
- **AI:** Hybrid.
- **Workflow:** Inspect metadata, suggest categories and save user corrections.
- **Exceptions:** Keep uncertain items Uncategorized; retain file-type information if the model fails.

F03. Find Relationships and Map Materials

- **Description:** Group related assets and bind textures or materials to models.
- **GUI:** Relationships and Material Mapping.
- **CLI:** analyze relationships or analyze materials.
- **Input:** Assets and existing references.
- **Output:** Reviewed groups and material mappings.
- **AI:** Hybrid.
- **Workflow:** Read references, suggest uncertain matches and add accepted mapping changes to the plan.
- **Exceptions:** Missing or ambiguous matches require correction or exclusion before migration.

F04. Rename and Organize Assets

- **Description:** Apply consistent names and an output folder structure.
- **GUI:** Rename and Organize preview.
- **CLI:** analyze names or analyze folders.
- **Input:** Inventory and naming rules.
- **Output:** Proposed names and layout, then organized output copies.
- **AI:** Hybrid.
- **Workflow:** Suggest changes, check destinations, obtain approval and execute them.
- **Exceptions:** Block duplicate names, invalid paths and unresolved references.

F05. Check Compatibility

- **Description:** Check assets against a target profile and installed converters.
- **GUI:** Check Compatibility.
- **CLI:** compatibility.
- **Input:** Inventory and target.
- **Output:** Issues and supported conversion choices.
- **AI:** Hybrid.
- **Workflow:** Apply format and reference checks, with optional AI explanations.
- **Exceptions:** Flag unknown requirements and unavailable tools; retain findings if explanation fails.

F06. Create and Approve a Migration Plan

- **Description:** Build an ordered plan for review.
- **GUI:** Generate Plan and Approve.
- **CLI:** plan and approve.
- **Input:** Target, reviewed changes and compatibility findings.
- **Output:** Versioned plan and approval state.
- **AI:** Hybrid.
- **Workflow:** Agent proposes steps; controller checks tools, dependencies and conflicts; user approves the current version.
- **Exceptions:** Block unsupported steps and stale approval; editing the plan requires renewed approval.

F07. Convert and Migrate Assets

- **Description:** Execute approved conversion and file changes.
- **GUI:** Start Migration.
- **CLI:** migrate.
- **Input:** Approved plan and output folder.
- **Output:** Prepared assets and step results.
- **AI:** Hybrid: the plan is AI-assisted; approved file operations are deterministic.
- **Workflow:** Recheck approval and source files, execute commands, stream progress and save results.
- **Exceptions:** Stop dependent work after a failed step; preserve sources and record partial or interrupted runs.

F08. Validate Results and View Reports

- **Description:** Check outputs and retain migration history.
- **GUI:** Validate Migration and History.
- **CLI:** validate and history.
- **Input:** Source inventory and recorded outputs.
- **Output:** Validation findings, status and report.
- **AI:** Hybrid.
- **Workflow:** Check file existence, readability, formats and references; save a report with an optional AI summary.
- **Exceptions:** Mark failed or unsupported checks clearly; use a deterministic summary if AI fails; report missing history.

F09. Natural-Language Commands

- **Description:** Access supported operations through plain language.
- **GUI:** Agent command panel.
- **CLI:** command.
- **Input:** User request and project context.
- **Output:** Action, result, draft plan or clarification.
- **AI:** AI-based.
- **Workflow:** Interpret an allowed action and dispatch it through AssetManager.
- **Exceptions:** Clarify ambiguous requests and reject unsupported actions; file changes still require an approved plan.

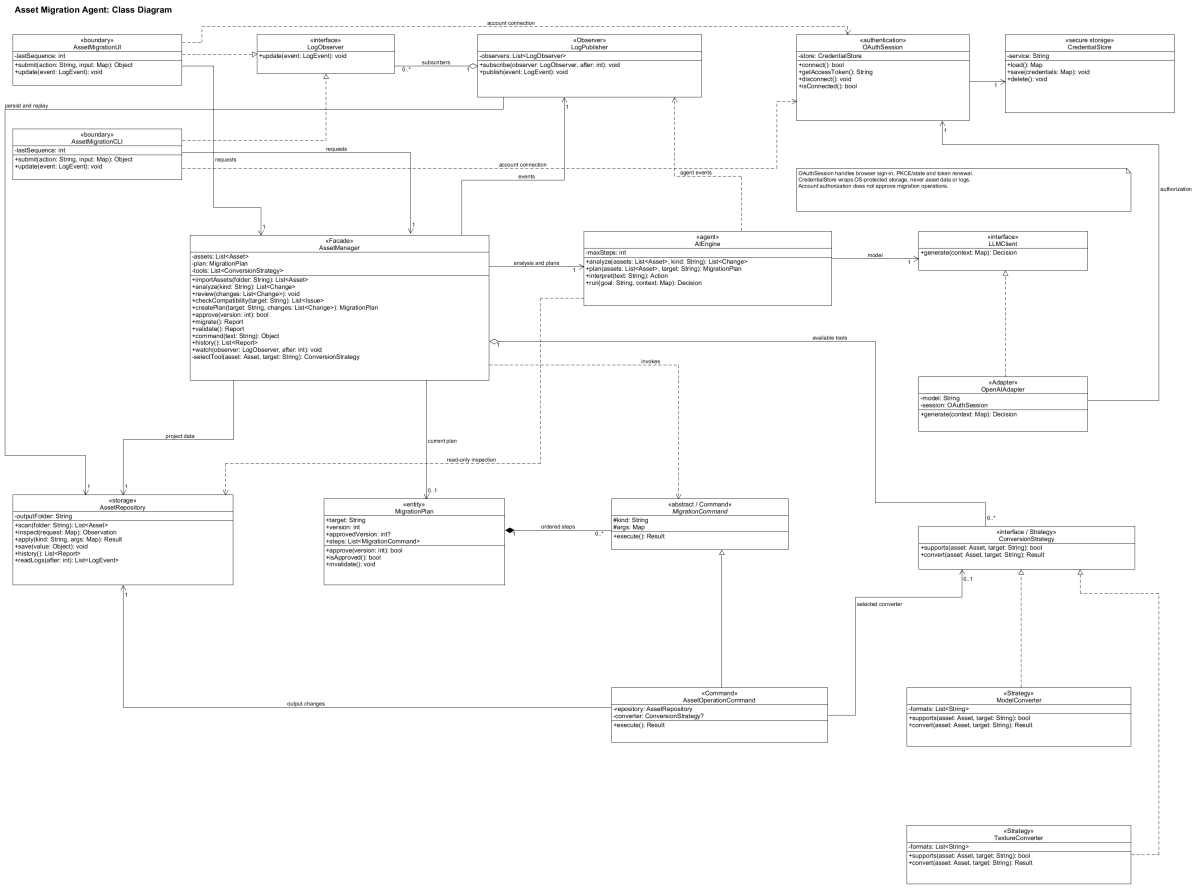
F10. Live Logs and Replay

- **Description:** Observe current work and review earlier events.
- **GUI:** Live Logs.
- **CLI:** logs.
- **Input:** Project and last received event number.
- **Output:** Ordered activity and progress events.
- **AI:** Deterministic.
- **Workflow:** Persist each event, update subscribers and replay missed events after reconnection.

- **Exceptions:** A disconnected viewer does not stop the worker; a logging failure is reported and new write steps stop safely.

2. UML Design

2.1 Class Diagram



Design Patterns

Facade. AssetManager provides one interface to agent, storage and conversion workflows. The GUI and CLI are its clients. This keeps approval and execution rules consistent; without it, both interfaces would duplicate workflow coordination.

Command. MigrationCommand is the abstract command, AssetOperationCommand is its concrete implementation, MigrationPlan stores commands and AssetManager invokes them. The repository and converters perform the changes. Capturing operations as objects makes them reviewable and ordered; without it, plans and execution logic would be harder to keep consistent.

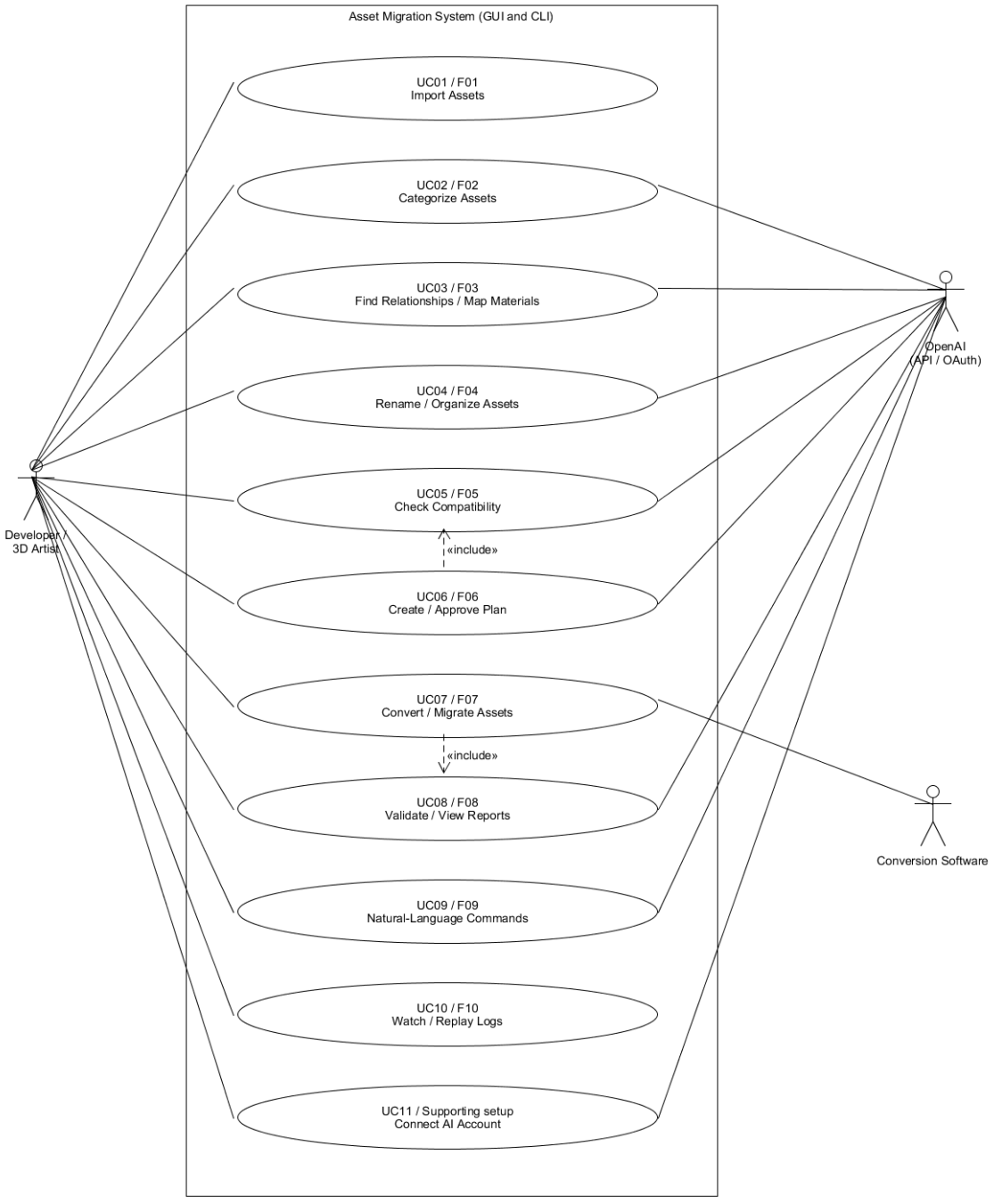
Strategy. ConversionStrategy defines the conversion contract. ModelConverter and TextureConverter implement different algorithms; AssetManager selects a supported converter for a command. This allows converters to change independently of the workflow. Without it, format-specific branches would spread through the controller.

Adapter. LLMClient is the application interface, OpenAIAdapter translates its requests and responses, and AIEngine is the client. The external service is the OpenAI API. This isolates provider details; without it, agent logic would depend directly on API payloads and errors.

Observer. LogPublisher is the subject, LogObserver defines the callback, and the GUI and CLI are observers. Workers publish events while operating. This supports interchangeable live displays; without it, workers would need direct knowledge of each interface.

2.2 Use-Case Diagram

Asset Migration Agent: Use-Case Diagram



Use-Case Descriptions

AI requests require authorized model access through UC11. Account authorization never grants migration approval.

UC01. Import Assets

- **Actors:** User.
- **Goal:** Create an inventory.

- **Preconditions:** Application running and folder selected.
- **Trigger:** Import Project or import.
- **Main scenario:** Select folder, scan supported files, extract metadata and save/display inventory.
- **Alternatives:** Reject inaccessible folders; flag skipped files.
- **Postconditions:** Inventory and warnings are available; sources are preserved.
- **Related feature:** F01.

UC02. Categorize Assets

- **Actors:** User, OpenAI API.
- **Goal:** Assign useful categories.
- **Preconditions:** Imported inventory.
- **Trigger:** Categorize Assets or analyze categories.
- **Main scenario:** Agent reviews metadata, suggests categories, and the user accepts or corrects them.
- **Alternatives:** Uncertain assets remain Uncategorized; model failure preserves type information.
- **Postconditions:** Reviewed categories are saved and affected approval is invalidated.
- **Related feature:** F02.

UC03. Find Relationships and Map Materials

- **Actors:** User, OpenAI API.
- **Goal:** Identify groups and correct material references.
- **Preconditions:** Relevant assets imported.
- **Trigger:** Relationship or mapping analysis.
- **Main scenario:** Inspect references, suggest matches, review mappings and add accepted output changes to a plan. Approved migration applies them.
- **Alternatives:** Missing or uncertain matches require correction or exclusion.
- **Postconditions:** Accepted relationships are saved and mappings are planned or applied.
- **Related feature:** F03.

UC04. Rename and Organize Assets

- **Actors:** User, OpenAI API.
- **Goal:** Produce consistently named and organized outputs.
- **Preconditions:** Inventory and naming rules available.
- **Trigger:** Rename/Organize or corresponding analysis command.
- **Main scenario:** Suggest names and folders, review changes, check conflicts and execute after plan approval.
- **Alternatives:** Block invalid or duplicate destinations; allow revision.
- **Postconditions:** Reviewed changes are saved and approved output copies are organized.
- **Related feature:** F04.

UC05. Check Compatibility

- **Actors:** User, OpenAI API for optional explanations.

- **Goal:** Identify migration problems.
- **Preconditions:** Inventory and target selected.
- **Trigger:** Compatibility check or plan generation.
- **Main scenario:** Check target rules and tool capabilities, then display findings.
- **Alternatives:** Flag unknown checks and missing tools; AI failure does not remove findings.
- **Postconditions:** Compatibility results are available.
- **Related feature:** F05.

UC06. Create and Approve a Migration Plan

- **Actors:** User, OpenAI API.
- **Goal:** Establish reviewed migration steps.
- **Preconditions:** Inventory, target and output folder selected.
- **Trigger:** Generate Plan or plan.
- **Main scenario:** Check compatibility, obtain proposed steps, validate commands, review the draft and approve its version.
- **Alternatives:** Block conflicts or unsupported steps; model failure allows retry; changes invalidate approval.
- **Postconditions:** A draft or approved plan is saved.
- **Related feature:** F06.

UC07. Convert and Migrate Assets

- **Actors:** User, Conversion Software.
- **Goal:** Execute approved work.
- **Preconditions:** Current approved plan, supported tools and writable output.
- **Trigger:** Start Migration or migrate.
- **Main scenario:** Recheck inputs, execute ordered commands, stream progress, validate outputs and save a report.
- **Alternatives:** Block stale plans; stop dependent work after failure; mark interrupted runs incomplete.
- **Postconditions:** Outputs and a success, partial, failed or incomplete report exist.
- **Related feature:** F07.

UC08. Validate Results and View Reports

- **Actors:** User, OpenAI API for optional summaries.
- **Goal:** Check and retrieve outcomes.
- **Preconditions:** An attempted run or saved report.
- **Trigger:** Migration completion, Validate Migration or History.
- **Main scenario:** Check recorded outputs, save findings and display the report; history loads earlier reports.
- **Alternatives:** Flag missing outputs or reports and unsupported checks; use a deterministic summary if AI fails.
- **Postconditions:** Results and completion status are retrievable.
- **Related feature:** F08.

UC09. Natural-Language Commands

- **Actors:** User, OpenAI API.
- **Goal:** Request supported operations in plain language.
- **Preconditions:** Project context and configured model access.
- **Trigger:** Agent-panel request or command.
- **Main scenario:** Interpret the request, validate its action, call the matching controller operation and display the result.
- **Alternatives:** Ask for clarification, reject unsupported actions or report model failure.
- **Postconditions:** A result, draft or clarification is returned under normal approval rules.
- **Related feature:** F09.

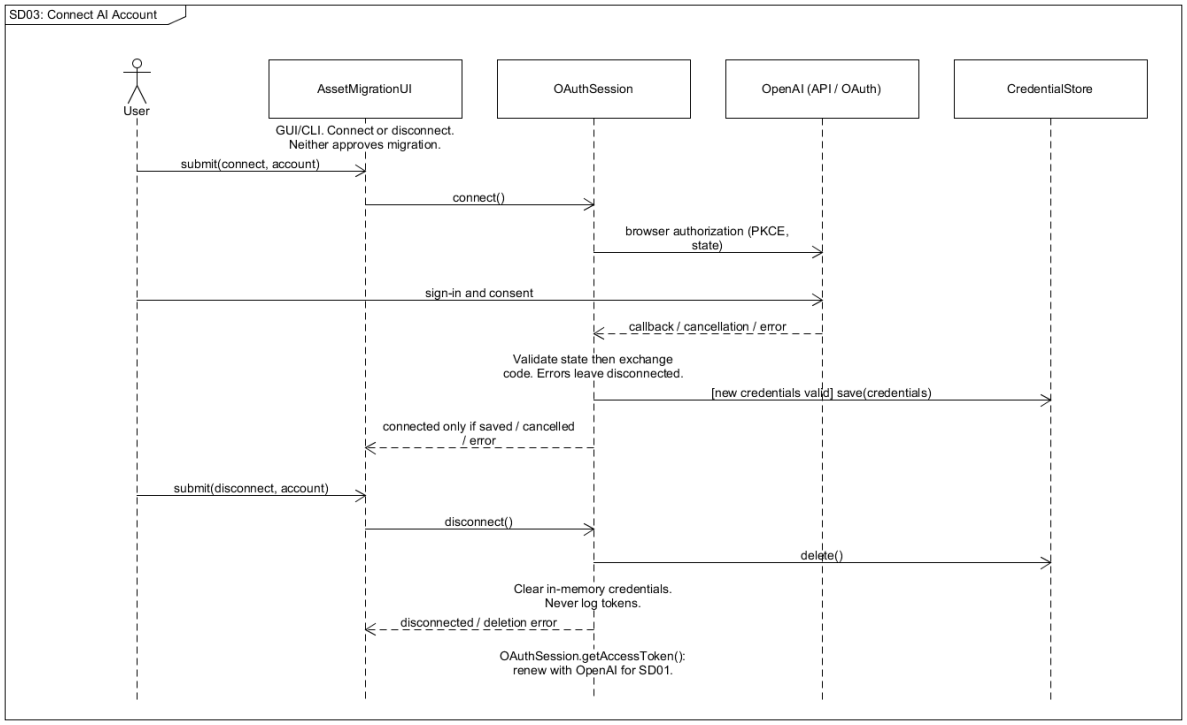
UC10. Live Logs and Replay

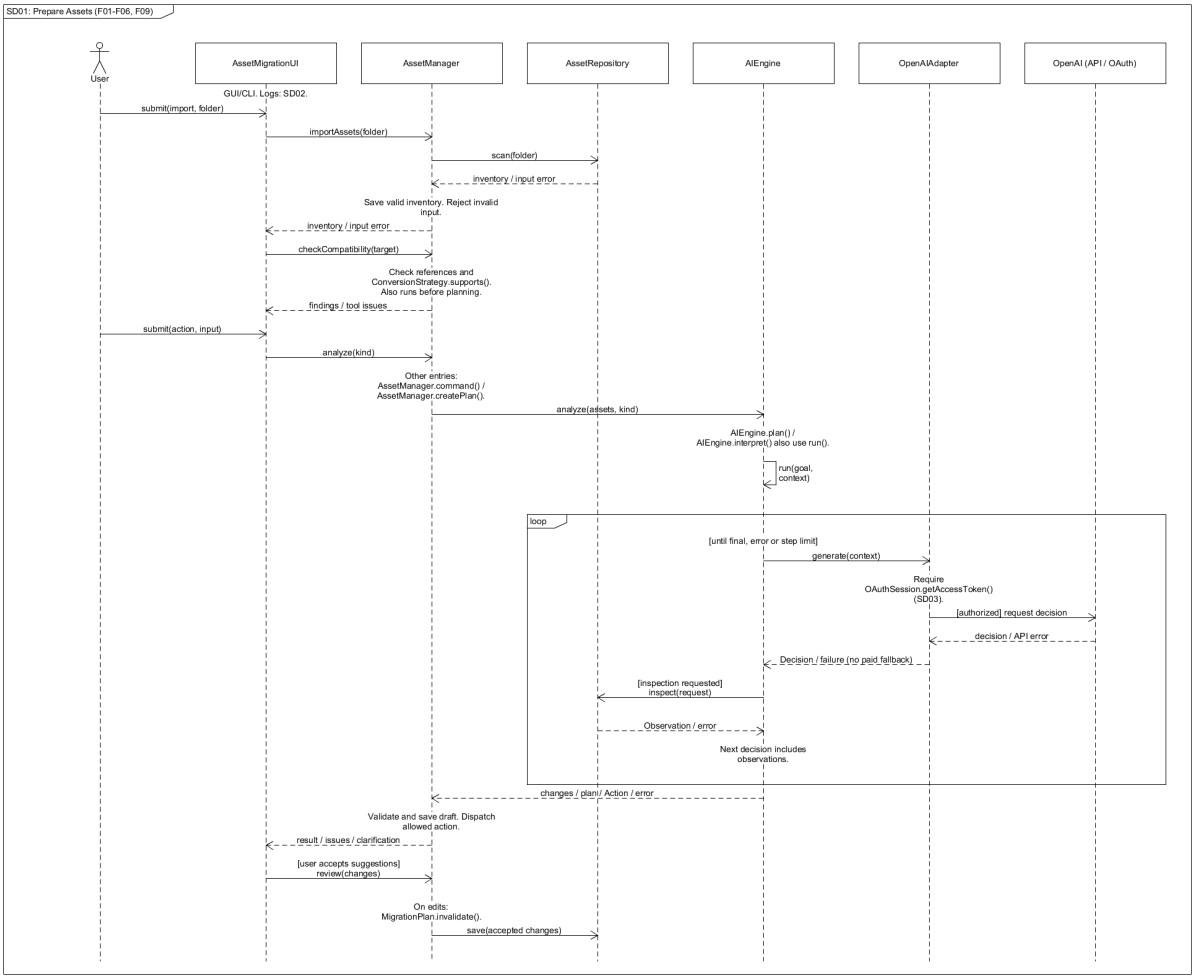
- **Actors:** User.
- **Goal:** Observe work and recover missed events.
- **Preconditions:** Readable project event store.
- **Trigger:** Live Logs or logs.
- **Main scenario:** Subscribe from a sequence number, replay saved events and follow new events.
- **Alternatives:** Reconnect after disconnection; stop new write steps safely if events cannot be saved.
- **Postconditions:** An ordered activity stream and replay cursor are available.
- **Related feature:** F10.

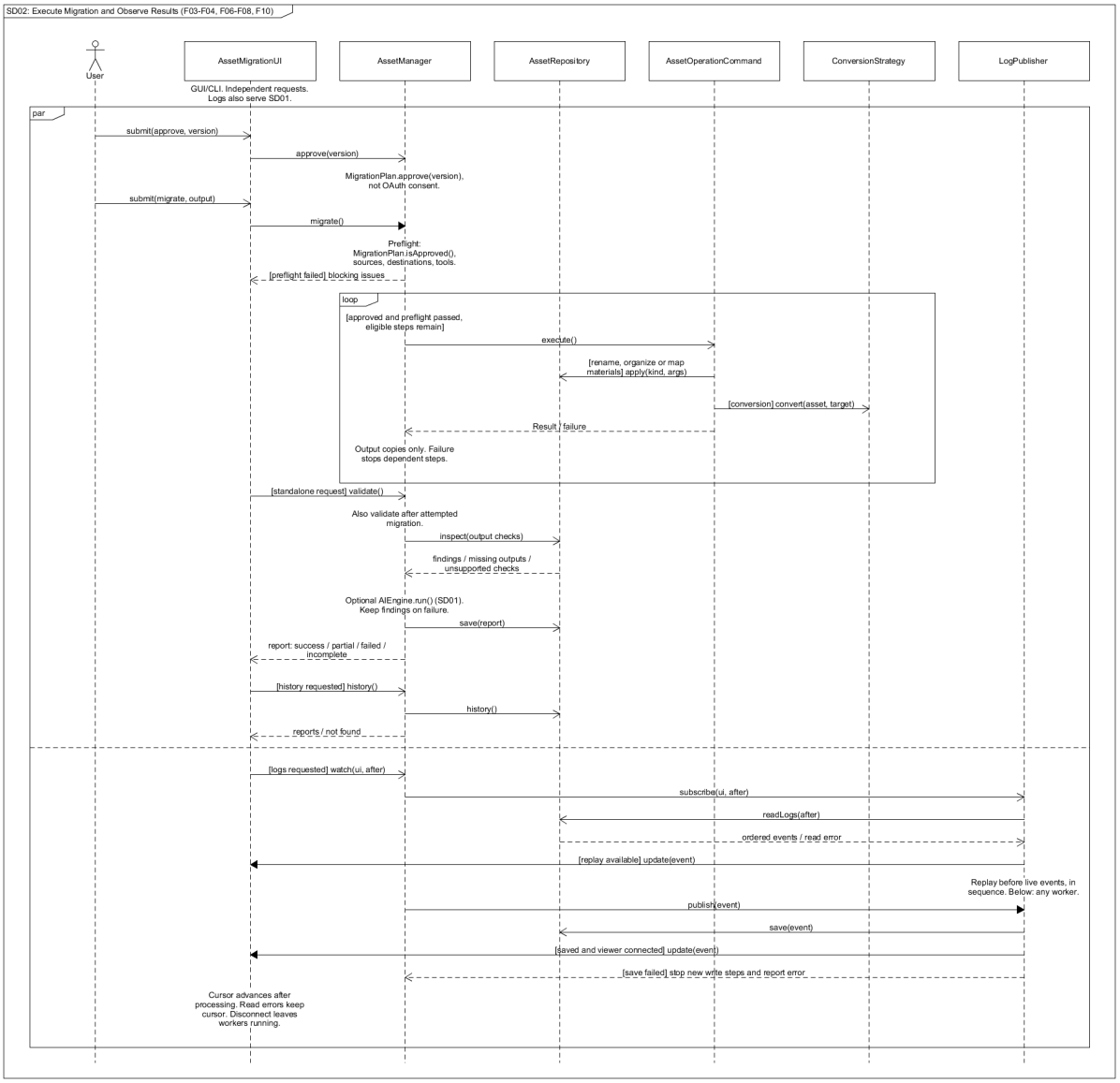
UC11. Connect AI Account (Supporting Setup)

- **Actors:** User, OpenAI authorization service.
- **Goal:** Authorize supported model usage.
- **Preconditions:** Network access and a provider-supported application registration and account.
- **Trigger:** Connect AI Account or auth connect.
- **Main scenario:** The interface calls OAuthSession.connect(), opens the provider's authorization page with PKCE and state, and the user signs in and consents. OAuthSession validates the callback, exchanges the code and saves credentials through CredentialStore. OpenAIAdapter obtains or renews access through getAccessToken() before model requests.
- **Alternatives:** Cancellation, invalid callbacks, exchange or storage failures do not establish a connection; expired or revoked access may require reconnection. Usage limits stop AI requests without paid fallback. Disconnect calls disconnect() and clears credentials without cancelling the subscription.
- **Postconditions:** Connection status is shown; migration approval is unchanged.
- **Related features:** Supporting setup for F02-F06, F08 and F09; not an additional core feature.

2.3 Sequence Diagram


SD03: Connect AI Account
User
AssetMigrationUI
OAuthSession
OpenAI (API / OAuth)
CredentialStore
GUI/CLI. Connect or disconnect.
Neither approves migration.
submit(connect, account)
connect()
browser authorization (PKCE, state)
sign-in and consent
callback / cancellation / error
Validate state then exchange code. Errors leave disconnected.
[new credentials valid] save(credentials)
connected only if saved / cancelled / error
submit(disconnect, account)
disconnect()
delete()
Clear in-memory credentials.
Never log tokens.
disconnected / deletion error
OAuthSession.getAccessToken():
renew with OpenAI for SD01.


SD01: Prepare Assets (F01-F06, F09)
User
AssetMigrationUI
AssetManager
AssetRepository
AIEngine
OpenAIAdapter
OpenAI (API / OAuth)
GUI/CLI. Logs: SD02.
submit(import, folder)
importAssets(folder)
scan(folder)
inventory / input error
Save valid inventory. Reject invalid input.
inventory / input error
checkCompatibility(target)
Check references and ConversionStrategy.supports(). Also runs before planning.
findings / tool issues
submit(action, input)
analyze(kind)
Other entries: AssetManager.command() / AssetManager.createPlan().
analyze(assets, kind)
AIEngine.plan() / AIEngine.interpret() also use run().
run(goal, context)
loop
[until final, error or step limit]
generate(context)
Require OAuthSession.getAccessToken() (SD03).
[authorized] request decision
decision / API error
Decision / failure (no paid fallback)
[inspection requested] inspect(request)
Observation / error
Next decision includes observations.
changes / plan / Action / error
Validate and save draft. Dispatch allowed action.
result / issues / clarification
[user accepts suggestions] review(changes)
On edits: MigrationPlan.invalidate().
save(accepted changes)



3. Feature-to-Design Traceability

Feature	Description	Type	Use Case	Classes	Key Methods	Sequence	Design Patterns
F01	Import assets	Deterministic	UC01	AssetManager, AssetRepository	importAssets(), scan(), save()	SD01	Facade
F02	Categorize assets	Hybrid	UC02	AssetManager, AIEngine, OpenAIAdapter	analyze(), run(), generate(), review()	SD01	Facade, Adapter
F03	Map relationships and materials	Hybrid	UC03	AssetManager, AIEngine, AssetRepository, AssetOperationCommand	analyze(), inspect(), review(), execute(), apply()	SD01	Adapter, Command

Feature	Description	Type	Use Case	Classes	Key Methods	Sequence	Design Patterns
F04	Rename and organize	Hybrid	UC04	AssetManager, AIEngine, MigrationPlan, AssetOperationCommand, AssetRepository	analyze(), createPlan(), execute(), apply()	SD01	Facade, Command
F05	Check compatibility	Hybrid	UC05	AssetManager, AssetRepository, ConversionStrategy	checkCompatibility(), inspect(), supports()	SD01	Facade, Strategy
F06	Create and approve plan	Hybrid	UC06	AssetManager, AIEngine, MigrationPlan	createPlan(), plan(), approve(), invalidate()	SD01	Facade, Adapter, Command
F07	Convert and migrate	Hybrid	UC07	AssetManager, MigrationPlan, AssetOperationCommand, ConversionStrategy	migrate(), isApproved(), execute(), convert()	SD01	Command, Strategy, Observer
F08	Validate and view reports	Hybrid	UC08	AssetManager, AssetRepository, AIEngine	validate(), inspect(), save(), history(), run()	SD01	Facade, Adapter
F09	Natural-language commands	AI-based	UC09	AssetManager, AIEngine, OpenAIAdapter	command(), interpret(), run(), generate()	SD01	Facade, Adapter
F10	Live logs and replay	Deterministic	UC10	AssetManager, LogPublisher, LogObserver, AssetRepository	watch(), subscribe(), publish(), readLogs(), update()	SD01	Observer

4. Feature Realization

F01-F10 correspond to UC01-UC10 respectively and share sequence SD01.

F01 Realization

AssetManager.importAssets() calls AssetRepository.scan(), stores the inventory through AssetRepository.save() and returns detected assets and warnings.

F02 Realization

AIEngine.analyze() uses the model and inspection loop to suggest categories. AssetManager.review() saves accepted metadata.

F03 Realization

AssetRepository.inspect() supplies references to AIEngine. Reviewed groups are saved; AssetOperationCommand.execute() applies approved material mappings.

F04 Realization

AIEngine suggests names and folders. MigrationPlan stores corresponding commands, which call AssetRepository.apply() after approval.

F05 Realization

AssetManager.checkCompatibility() combines repository inspection with converter support checks. Findings remain available if optional explanation fails.

F06 Realization

AIEngine.plan() proposes steps. AssetManager creates and checks commands; MigrationPlan.approve() records approval and invalidate() clears it after edits.

F07 Realization

AssetManager.migrate() verifies approval and invokes commands. ConversionStrategy implementations perform supported conversions and publish progress.

F08 Realization

AssetManager.validate() checks outputs through the repository, saves a Report and optionally requests an AI summary. AssetManager.history() retrieves saved outcomes.

F09 Realization

AssetManager.command() uses AIEngine.interpret() to obtain an allowed action and dispatches the same operations used by explicit controls.

F10 Realization

LogPublisher persists and delivers events through LogObserver.update(). Saved sequence numbers support replay in either interface.